

from Beginner to Master

DevOps를 위한 **Docker** 가상화

Section 3. Docker Network and Storage

```
book:
    def setData(self, title, price, author):
        self.title = title
        self.price = price
        self.author = author

var fs = require('fs');
fs.readFile('./ONE.txt', '* 1 *',
    function (err, data) {
        console.log(data); // 3
    });
ApplicationDelegate > {
```

```
public static void main(String[] args)

<servlet>
    <servlet-name>BoardController</servlet-name>
    <servlet-class>com.joneconsulting.controller.BoardCont
    <init-param>
        <param-name>user_name</param-name>
        <param-value>henmeth Lee</param-value>
    </init-param>
    </servlet>
```

Winterface

Dowon Lee

수강생 19K 강의평점 4.8(1K)



멘토링 ON

멘토링 설정

- 홈
- 강의
- 로드맵
- 수강평
- 게시글
- 블로그

수정하기

강의 전체 6



[개정판] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정

▶ 학습중

★ 0강 / 25강 (0%)

818명



멀티OS 사용을 위한 가상화 환경 구축 가이드 (Docker + Kubernetes)

▶ 학습중

★ 0강 / 16강 (0%)

924명



Jenkins를 이용한 CI/CD Pipeline 구축

▶ 학습중

★ 81강 / 83강 (98%)

독점 2709명

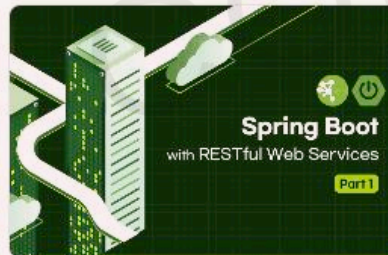


Spring Cloud로 개발하는 마이크로서비스 애플리케이션(MSA)

▶ 학습중

★ 156강 / 156강 (100%)

독점 5531명



Spring Boot를 이용한 RESTful Web Services 개발

▶ 학습중

★ 3강 / 48강 (6%)

독점 3862명



[구버전] 웹 애플리케이션 개발을 위한 IntelliJ IDEA 설정 (2020 ver.)

▶ 학습중

★ 14강 / 14강 (100%)

4813명

이 되었던 때가 있었습니
"IT 엔지니어"라고 적고

을 가지고 있습니다. 누구
사람입니다. 개발자로서, 강
지만, 그래도, 남들보다 조

ecture, Blockchain,

- Section 0: Introduction to Cloud Native Technologies
- Section 1: Docker Essentials - Container
- Section 2: Docker Essentials - Image
- **Section 3: Docker Network and Storage**
- Section 4: Building and Managing Containerized Application
- Section 5: Container Orchestration
- Section 6: Continuous Integration and Continuous Deployment
- Section 7: Docker Security
- Section 8: Logging and Monitoring
- Section 9: Advanced Docker Usage
- Section 10: Capstone Project

Section 3.

Docker Network and Storage

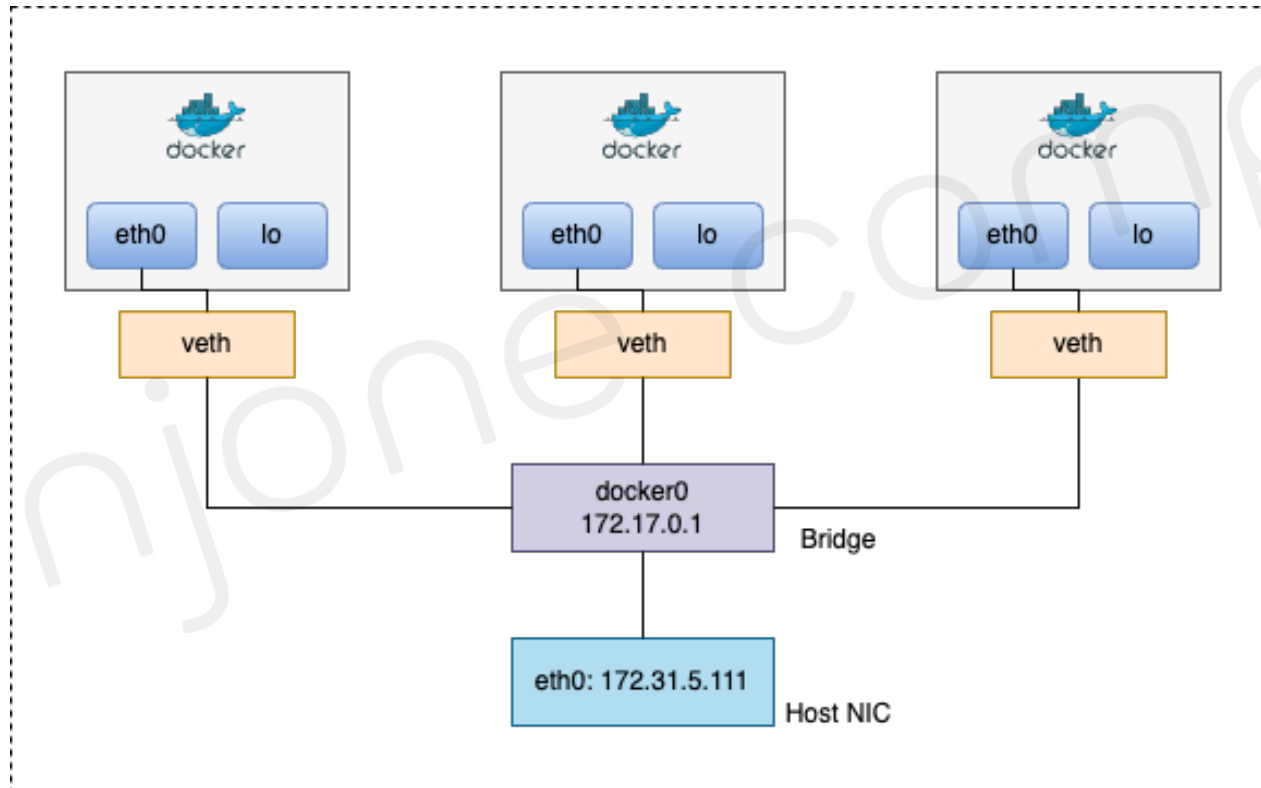
- Docker Network
- Multi Container
- Docker Volume Storage

컨테이너 간 통신을 위한 Docker Network 사용

- Container Isolation

- | Docker Container는 격리 된 환경에서 작동 → 다른 Container와 통신 불가능

- Docker Network



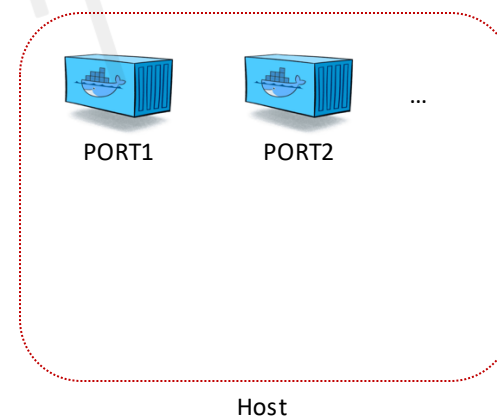
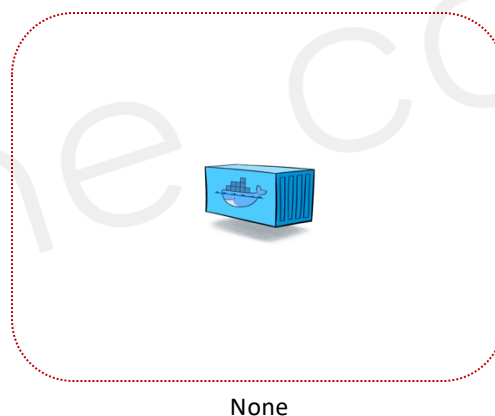
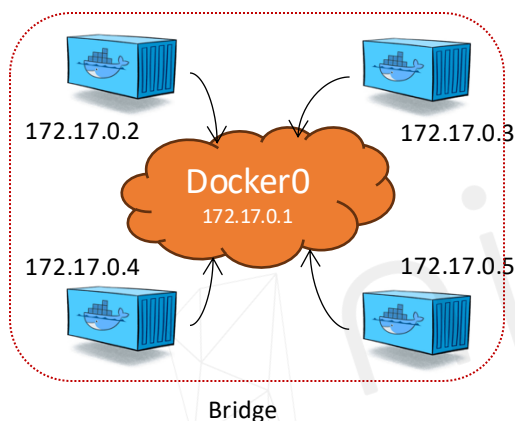
Docker Network

컨테이너 간 통신을 위한 Docker Network 사용

■ Docker Network 종류

- | Bridge Network : 하나의 Host 컴퓨터 내에서 여러 컨테이너들이 서로 통신 가능
- | Host Network : Host 컴퓨터와 동일한 네트워크에서 컨테이너를 기동
: 포트 포워딩 없이 내부 애플리케이션 사용
- | None Network : 네트워크를 사용하지 않음, lo 네트워크만 사용, 외부와 단절

Single Host Network



- | Overlay Network : 여러 호스트에 분산되어 컨테이너 실행 가능

Multi Host Network

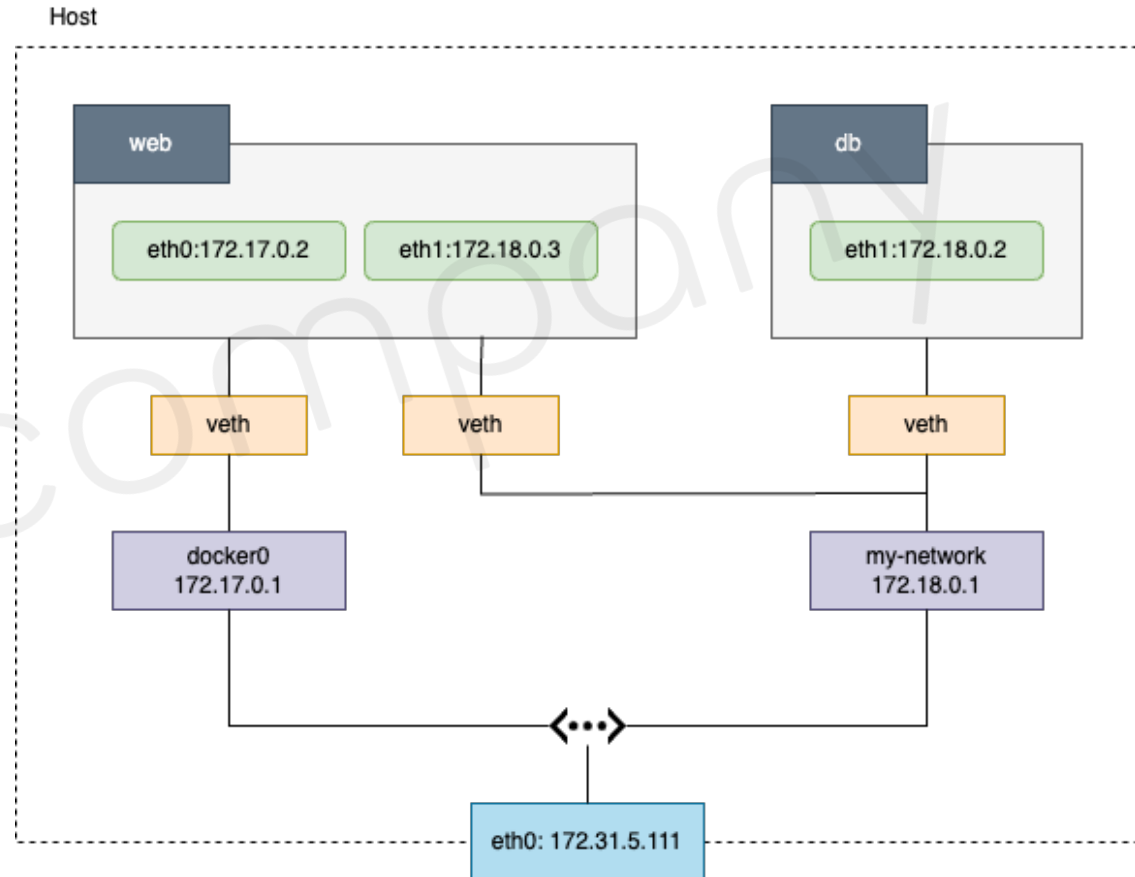
Docker Network 구성

- 기본 Docker Network → docker0

```
br-2b280c2e3756: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    ether 02:42:ad:2c:73:09 txqueuelen 0 (Ethernet)
    RX packets 176301 bytes 18705834 (17.8 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 269757 bytes 3913548966 (3.6 GiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet6 fe80::42:54ff:fe7d:15ae prefixlen 64 scopeid 0x20<link>
    ether 02:42:54:7d:15:ae txqueuelen 0 (Ethernet)
    RX packets 176301 bytes 18705834 (17.8 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 269757 bytes 3913548966 (3.6 GiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001
    inet 172.31.5.111 netmask 255.255.240.0 broadcast 172.31.15.255
    inet6 fe80::ad:14ff:fed5:9033 prefixlen 64 scopeid 0x20<link>
    ether 02:ad:14:d5:90:33 txqueuelen 1000 (Ethernet)
    RX packets 3953897 bytes 4921915174 (4.5 GiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1039629 bytes 1779811181 (1.6 GiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```



Docker Network 명령어

- Docker Container에서 사용하기 위한 Network 생성 및 관리

Command	설명
docker network ls	Docker Network 목록 보기
docker network inspect <NETWORK_NAME>	Docker Network 상세 보기
docker network create <NETWORK_NAME>	Docker Network 생성
docker network rm <NETWORK_NAME>	Docker Network 삭제
docker network connect <NETWORK_NAME> <CONTAINER>	실행 중인 Container를 Docker Network에 추가
docker network disconnect <NETWORK_NAME> <CONTAINER>	Docker Network에 추가 된 Container를 삭제

```
$ docker network create --driver bridge <NETWORK_NAME>
```

```
$ docker network connect my-network my-container
```

```
$ docker run -it --name my-container --network <NETWORK_NAME> ubuntu:16.04
```


Docker Network Inspect

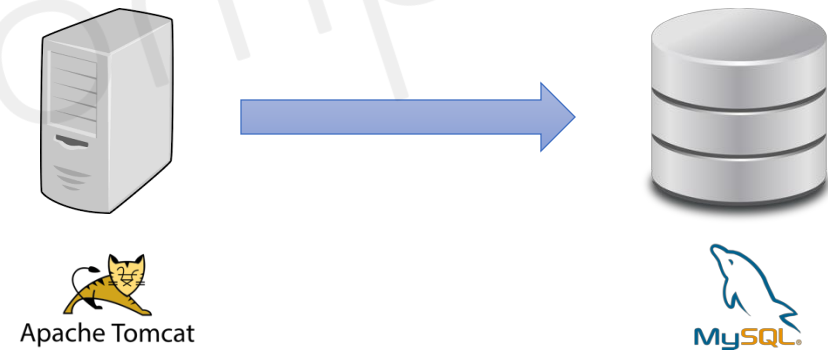
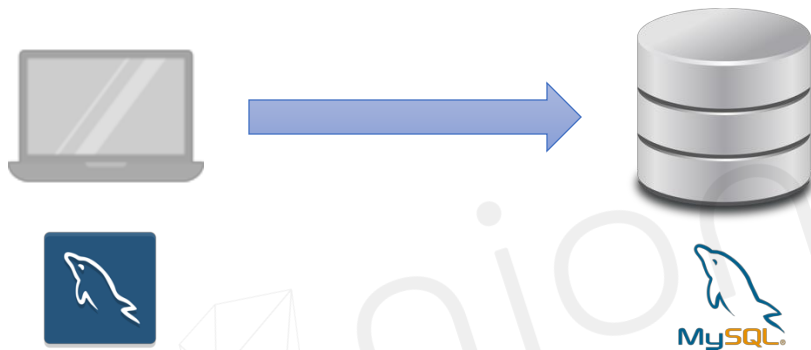
- Docker Network 상세 정보 확인

```
$ docker network inspect bridge
```

```
{
  "Name": "bridge",
  "Id": "63afb9cec9c2a5f9c93adde8e826bd3a6259a911f4342bf6af921fa5cab0cc53",
  "Created": "2024-02-05T22:32:48.880933958Z",
  "Scope": "local",
  "Driver": "bridge",
  "EnableIPv6": false,
  "IPAM": {
    "Driver": "default",
    "Options": null,
    "Config": [
      {
        "Subnet": "172.17.0.0/16",
        "Gateway": "172.17.0.1"
      }
    ]
  },
}
```

Docker Network 생성하기

- 실습) 다음과 같은 Docker Network를 구성하시오
 - | Mysql DB Client (Host PC) + Mysql DB Server (Docker)
 - | Tomcat (Host PC) + Mysql DB Server (Docker)

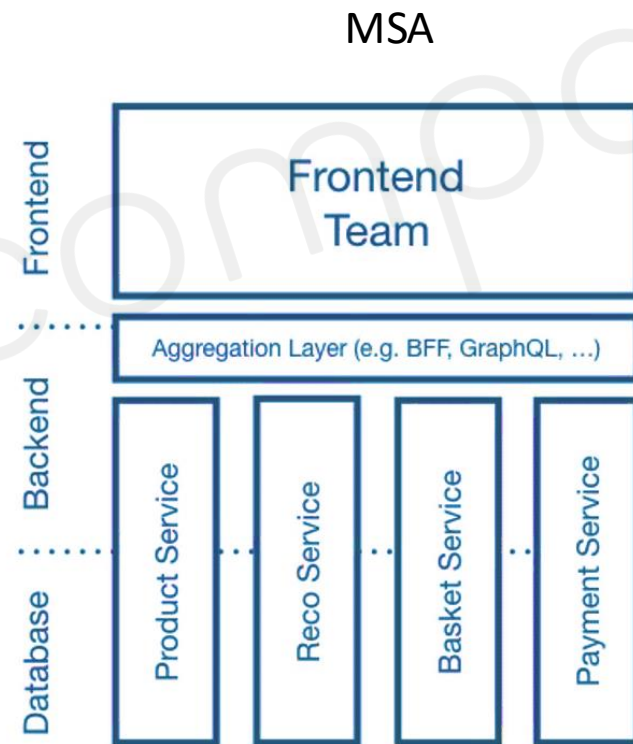
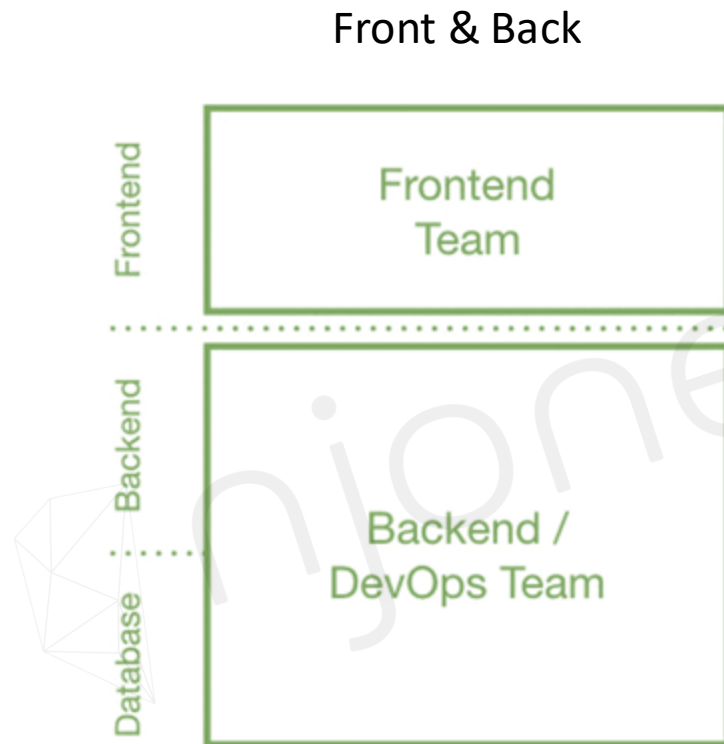


Multi Container

다중 컨테이너 구성

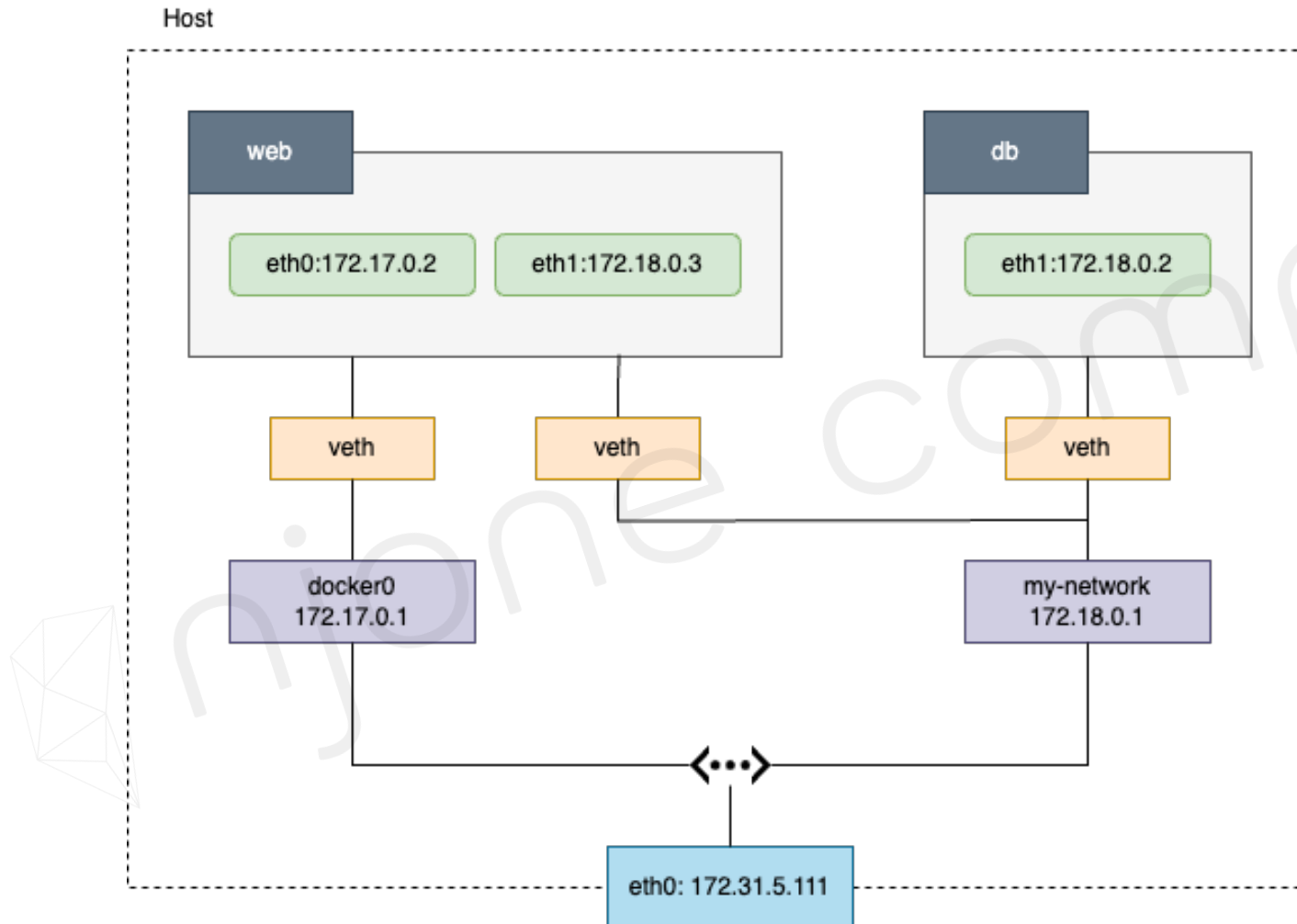
- 웹 3계층 시스템 아키텍처

| Frontend Server + Backend Server (Application Server) + Database



다중 컨테이너 구성

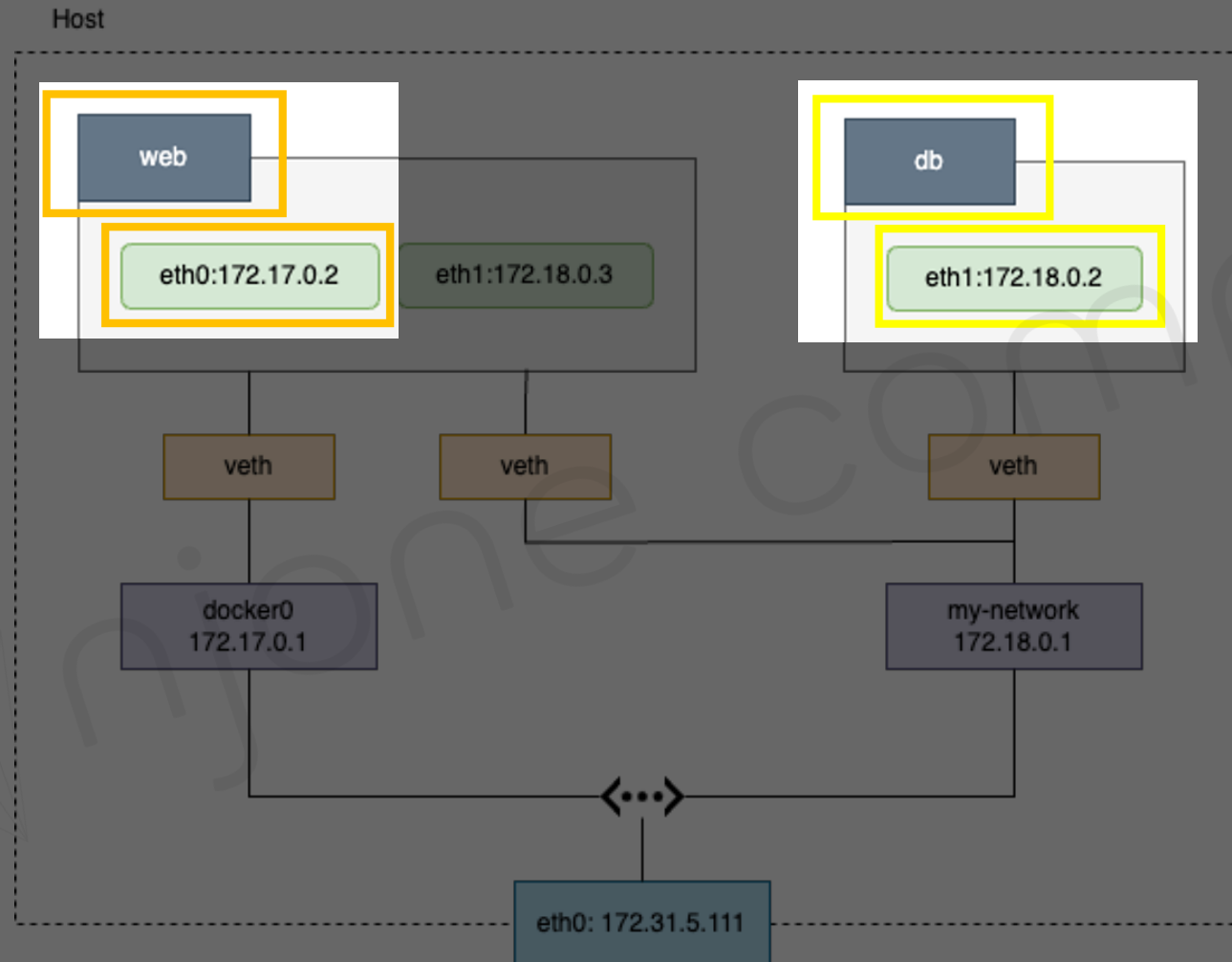
- 2개의 컨테이너 (web, db)에 독립적인 Docker Network 할당하여 구성



Multi Container

다중 컨테이너 구성

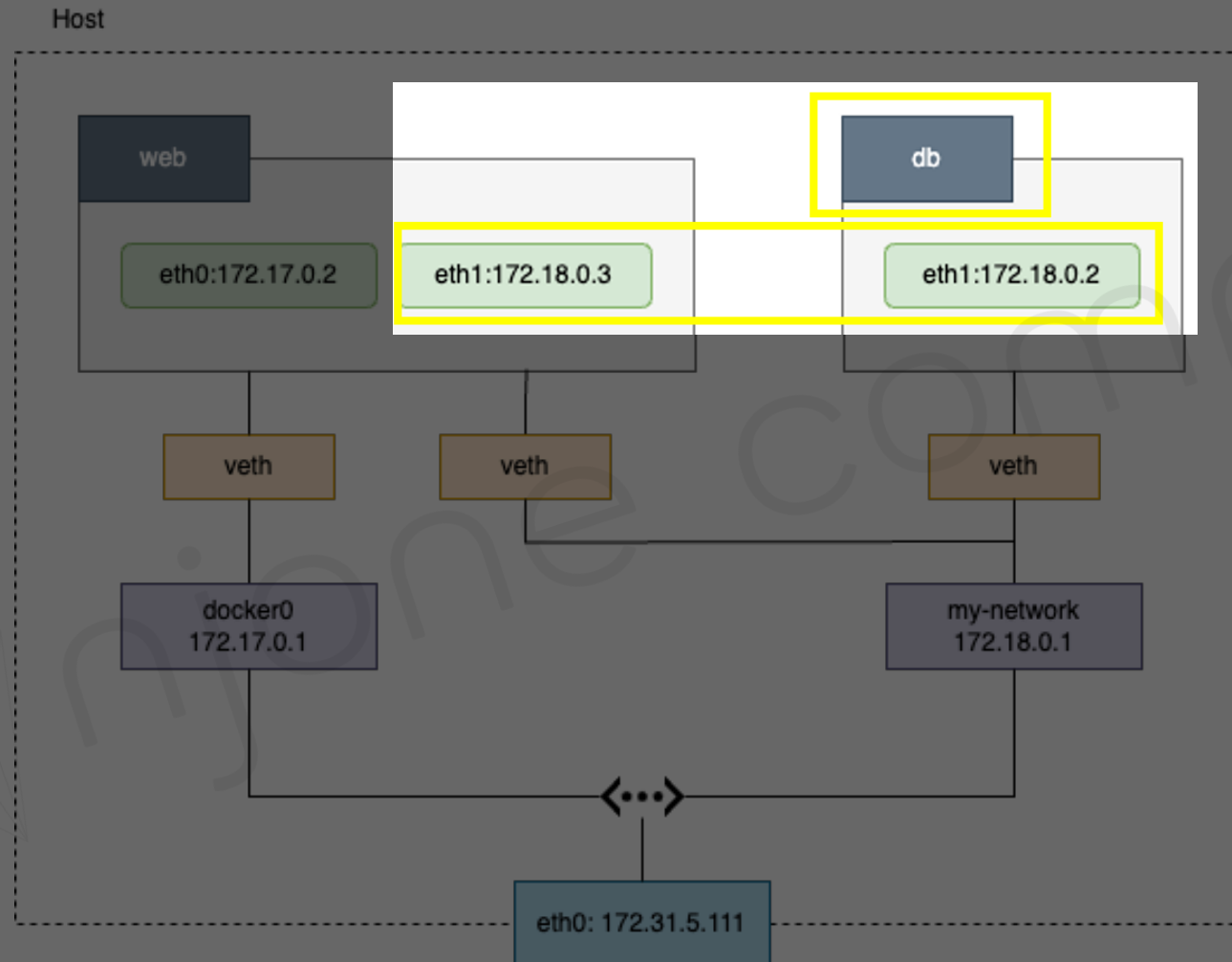
- 2개의 컨테이너 (web, db)에 독립적인 Docker Network 할당하여 구성



Multi Container

다중 컨테이너 구성

- 2개의 컨테이너 (web, db)에 독립적인 Docker Network 할당하여 구성



Multi Container

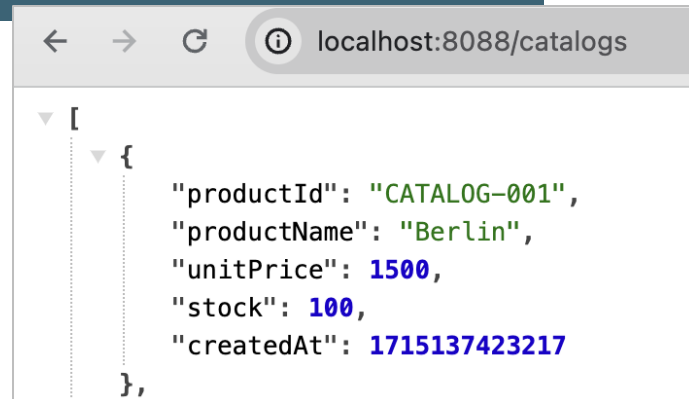
컨테이너 간 데이터 공유

- Backend Server → Database

```
$ docker network create --driver bridge my-network
```

```
$ docker run -d -p 13306:3306 --network my-network \  
-e MARIADB_ALLOW_EMPTY_ROOT_PASSWORD=true \  
-e MARIADB_DATABASE=mydb \  
--name my-mariadb edowon0623/my-mariadb:1.0
```

```
$ docker run -d -p 8088:8088 --network my-network \  
-e "spring.datasource.url=jdbc:mariadb://my-mariadb:3306/mydb" \  
--name catalog-service edowon0623/catalog-service:mariadb-demo
```



Docker Volume Storage

Docker Volume 이란?

- Docker는 개별적인 가상화 환경 → 모든 데이터는 컨테이너 내부에 존재
 - | 컨테이너가 삭제되면 작업했던 모든 데이터도 삭제
- 컨테이너 내부의 데이터를 외부로 연결 시켜 주는 기능
 - | Docker Volume은 기본적으로 /var/lib/docker/volumes/ (Docker Desktop) 디렉토리에 저장

```
$ docker volume create my-volume
```

```
$ docker volume inspect my-volume
```

- 컨테이너 생성 시 Volume 연결

```
$ docker run -v <볼륨이름> : <컨테이너 내부 디렉토리 경로> --name <컨테이너 이름> <이미지 이름>
```